Parallel External Sorting Revisited

Charles Benello*
University of Chicago
cmbenello@uchicago.edu

Riki Otaki*
University of Chicago
rotaki@uchicago.edu

Fuheng Zhao
UC Santa Barbara
fuheng_zhao@ucsb.edu

Aaron J. Elmore
University of Chicago
aelmore@cs.uchicago.edu

Goetz Graefe
Google
goetzg@google.com

Abstract

Sorting remains a fundamental operation in large-scale data systems. As data increasingly outgrows main memory, external merge sort is essential — but most implementations over-allocate memory and over-parallelize, inadvertently degrading performance by inflating buffer pressure and forcing extra merge passes. We present a quantile partitioned parallel external merge sort that self-configures from resource configurations, such as memory budget, available threads, and input size — to select memory size and to apportion parallelism between run generation and merging intelligently. The planner right-sizes buffers and parallelism to the memory budget, prefers single-pass merging when possible, and keeps partitions balanced via quantile cuts. We evaluate the configuration rules across TPC-H and synthetic inputs and on modern NVMe systems; ablations illustrate how misconfigurations (e.g., oversized runs or excessive merge parallelism) erode efficiency. We also describe an OVC-based sorting that reduces comparisons, cache misses, and I/O volume. The result is a configuration-first method that uses only the memory and parallelism the workload can productively exploit.

ACM Reference Format:
Charles Benello, Riki Otaki, Fuheng Zhao, Aaron J. Elmore, and Goetz Graefe. 2026. Parallel External Sorting Revisited. In . ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Sorting remains a critical performance bottleneck in modern data systems, from relational database engines and large-scale analytics pipelines to scientific computing workloads, often consuming a substantial fraction of total query execution time. It is used heavily in data loading [16, 29, 57], index creation [7, 28, 31, 32, 56, 66], and other data management tasks such as data deduplication and aggregation [8, 20, 25, 35, 68, 77], top-k queries [12, 30], and temporal joins [5, 11, 24, 26, 37, 65, 67, 80]. As data volumes increasingly exceed the capacity of main memory, external sorting techniques which involve transferring data between memory and external storage have become essential.

In modern cloud environments—where resources are shared and workloads are dynamic—it is crucial to design sorting algorithms that use as few resources as possible while still achieving high throughput [10, 14, 15, 23, 53, 54, 63, 70, 79, 81]. This efficiency is particularly vital given the recent economic shifts in hardware procurement; the explosion of Large Language Models (LLMs) has led to a global surge in demand for high-bandwidth and high-capacity memory, driving up prices and limiting availability for general-purpose computing. As main memory becomes an increasingly expensive and contended resource, the economic incentive shifts toward external algorithms that can maximize the utility of smaller memory footprints by efficiently leveraging lower-cost persistent storage.

External sorting has been extensively researched in terms of optimizing run generation algorithms [29, 48, 61], efficiently laying out memory during run generation [50, 51], balancing merge trees for reduced I/O during multi-pass merge [27, 43, 49, 69, 83], overlapping I/O and compute during merge [43, 69, 83], and optimizing merging algorithms using comparison caches from run generation [19, 20, 29, 33]. In addition, with the proliferation of multicore processors—parallelism in both run formation and merging has also been widely studied in the context of shared nothing architectures [4, 18, 25], cache conscious designs [61], skew tolerance [21, 40, 55], modern SSDs [60], and graceful degradation [48]. These prior works maximize performance under fixed memory and thread budgets, but do not consider how to optimally configure these resources for a given workload. A key challenge is that in a shared-memory setting, these resources are interdependent: increasing thread count reduces the memory available per thread, which in turn affects buffer sizes, run lengths, and merge depth—creating a joint configuration problem that cannot be solved by tuning each resource in isolation. In other words, they do not address when it is worth adding more memory or threads to speed up external sorting, and if so, how much of each, nor how to balance this allocation. This is the question we address in this work.

In practice, given an input dataset of size D, a sorter with memory budget M and with at most T threads, a naïve policy is to first max out T, split M into $\approx M/T$ per thread, and assign a subset of data to each thread; however, this policy creates two coupled problems: **over-memory** (allocating more than is needed for the target run length and merge plan) and **over-parallelism** (too many threads shrinking per-thread buffers, shortening runs, and deepening the merge tree). We address both jointly: right-size buffers to target long runs and (near) single-pass merging; then calibrate T to ensure

*These authors contributed equally to this work.

Unpublished working draft. Not for distribution.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Unpublished working draft. Not for distribution.

that we execute the external sort without deepening the merge tree. The outcome is a sorter that uses **no more memory and no more threads** than the workload can productively exploit. Moreover, to achieve optimal performance, it is desirable to load-balance the work and partition D as evenly as possible among the available threads. If the work is not partitioned evenly, threads will receive imbalanced loads, leading to idle time and reduced throughput. To this end, we employ Weighted Reservoir Sampling [22] and KLL sketches [39, 41] to approximate data distributions and identify quantile boundaries, yielding partitions that are near-optimally balanced with minimal sampling overhead.

In addition, recent advances in offset value coding (OVC) [19, 20, 29, 33, 34, 46] have shown that the number of comparisons can be significantly reduced during run generation and merge by caching the results of comparisons. Moreover, OVC aligns well with prefix truncation [32], which reduces I/O volume during run generation and merge. We study the impact of OVC on end-to-end external sorting performance and show that it consistently improves throughput on workloads with sort-keys sharing common prefixes. Finally, we introduce a lightweight quantile-based partitioning scheme for parallel merging that balances work across threads without fine-grained synchronization.

Contributions. This paper makes the following contributions:

- **Resource-aware planning for external sort.** We derive practical rules that jointly choose buffer size and apportion threads between run generation and merging, as a function of memory budget, core count, and input size, avoiding both *over-using memory* (buffer pressure, spill amplification) and *over-using threads* (over-parallelization, extra merge operations).
- **I/O and CPU-efficient sorting via Offset Value Codes (OVC).** We show that OVC reduces I/O volume and lowers comparison cost, yielding higher end-to-end throughput for external sorting workloads with sort-keys sharing long common prefixes.
- **Lightweight, quantile-guided parallel merging.** We introduce a simple quantile-based partitioner based on Weighted Reservoir Sampling [22] and KLL [39, 41] that balances merge work across threads without fine-grained synchronization, improving scalability while keeping coordination overhead minimal.

Paper Organization. Section 2 situates our work in the context of prior systems for parallel external sorting frameworks. Section 3 formalizes the design degrees of freedom—memory budgeting and parallelism—and derives practical configuration rules. Section 4 details the implementation of our external-sorting engine. Section 5 evaluates the resulting design across resource regimes, quantifies the impact of each knob, and identifies near-optimal configurations. Section 6 summarizes findings and outlines implications for practitioners. Finally, Section 7 concludes.

2 Background and Related Work

2.1 Foundations of External Sort

In order to sort datasets that exceed available memory, systems must process data in chunks that fit into memory, and then merge or concatenate these chunks to produce the final sorted output. There are two main paradigms for large-scale parallel sorting: *merge-based* and *distribution-based* approaches [27, 29, 43, 72]. Merge-based methods have two phases: (1) *run generation*, where sorted runs are generated and written to disk; and (2) *run merging*, where these runs are merged into a single globally sorted output. Distribution-based methods partition the key space into disjoint ranges, sort each partition, and concatenate the results directly. This avoids multi-way merging but depends heavily on accurate partition boundaries and is less robust to skewed distributions than merge-based methods [18, 29, 45, 78]. As a result, merge-based sorting remains the dominant approach in practice used by systems such as Unix sort [1], PostgreSQL [76], and DuckDB [48]. From here on, we focus on merge-based external sorting.

2.2 Run Generation Techniques

There are a variety of run generation techniques, each with different trade-offs in terms of run length, CPU cost, and I/O efficiency [29]. The two most common methods are: *load-sort-store* and *replacement selection*. In load-sort-store, data is read into memory, sorted using in-memory sorting algorithms (e.g., quicksort, radixsort), and then written back to disk. Replacement selection, on the other hand, uses a priority queue to produce runs that are twice the size of available memory on average [29, 43, 50, 51]. Replacement selection has lower cache locality than *load-sort-store* due to the use of priority queues but can overlap I/O and computation more effectively [50, 51, 61]. AlphaSort [61] optimized the cache efficiency of *load-sort-store* by sorting an array of key prefix and record pointer pairs, reducing memory bandwidth. More recently, DuckDB [48] employed radix sort for run generation, which incurs fewer branch mispredictions than quicksort-based methods, achieving high throughput on modern CPUs. They recently redesigned their sort pipeline in 2025 [47] and their in-memory sorting algorithm defaults to vergesort [58] with skasort [73, 74].

2.3 Merge Techniques

Merging sorted runs is the second phase of external merge sort. The performance of merging is primarily determined by the number of runs to be merged (merge degree or fan-in), which affects the total I/O volume [29, 43, 47, 48, 51, 64, 69, 82, 83]. When a single merge operation cannot merge all runs in one pass, multiple merge operations are required, causing parts of the data to be read and written to disk multiple times. For merging multiple runs, a priority queue (min-heap) or a tournament tree is typically used to efficiently select the next smallest element among the runs [29, 43]. Tree-of-losers is a tournament tree that minimizes key comparisons per output [29, 33, 43]. This data structure also complements offset value coding (OVC), which will be discussed more in Section 2.4. For optimizing I/O efficiency, techniques such as double buffering with asynchronous I/O [27, 43, 69, 83] and forecasting [43] have been proposed to overlap I/O and computation.

Unpublished working draft. Not for distribution.

2.4 Offset Value Coding

Offset-value coding (OVC) [6, 19, 20, 29, 33, 34] encodes a comparison as (o, v) where o is the index of the first differing byte and v is the byte from the non-reference key at that index. Two OVCs can be used to compare two keys if they were computed against the same reference key (order by offset first; break ties by the recorded byte). If OVCs are equal, the original keys from the offset onward must be compared to break the tie. In merge-sorts, OVCs can be emitted during run formation and reused during multiway merge; a tree-of-losers updates these codes efficiently along the leaf-to-root path [33]. OVC is effective for normalized keys, where sort keys are normalized to a byte-comparable layout [29, 48]. OVC also aligns well with prefix truncation in sorting: the offset o reveals the shared-prefix length with the preceding run element, informing safe truncation and reducing I/O during run generation and merging [32].

2.5 Parallelizing External Merge Sort

Parallel external sorting has been classified by how it parallelizes run generation and merging [75]. Run generation is often parallelized by dividing the input into equal-sized chunks and having each processor sort its chunk independently [21, 61, 75]. DuckDB [47, 48] employs morsel-driven parallelism [52] where each thread dynamically fetches morsels of data to sort. Merging can be parallelized in two ways: *horizontally*—merging the same set of runs with each thread assigned to different output ranges; or *vertically*—assigning different merge operations (different runs) to different threads. Horizontal partitioning requires identifying split points that balance work across threads. For two-way merges, optimal split points can be computed efficiently using the MergePath algorithm [47, 62]. For multi-way merges, optimal split points are typically approximated using sampling [3, 18, 40, 55], which will be described further in Section 2.6. Vertical partitioning produces multiple sorted runs that must be merged again to create a single global output.

2.6 Quantile-Based Range Partitioning

Quantile accuracy drives load balance in range-partitioned parallel merge. Random sampling is simple and cheap; a sample of size $O(1/\epsilon^2)$ guarantees (with constant probability) that each estimated quantile is within $\pm\epsilon$ of the truth [13, 22], and oversampling further improves balance [9]. For large data, compact summaries—histograms [17], data sketches [36, 71**? ?**], and KLL [42] with optimal $O(1/\epsilon)$ space [41]—provide efficient quantile approximations, providing high accuracy with fast update speed in one pass over the data. Learned partitioners can yield highly accurate equi-depth cuts on complex or skewed inputs [44], but they typically require significant pre-processing and relatively high memory overhead.

3 Memory- and Thread-Aware External Sorting

The prior section mapped the design space of parallel external sorting. While external sorting is well studied, there is little practical guidance on *jointly* choosing the memory budget and the degree of parallelism for a given input size, although these parameters are not independent and strongly couple the performance of the sort. This section derives simple, *actionable* constraints that couple memory

Table 1: Notation and parameters

Symbol	Description	Unit
D	Input size	bytes
M	Total memory budget	bytes
T_{gen}	Threads for run generation	count
T_{merge}	Threads for merge	count
$T_{\max}$	Hardware threads available	count
P	I/O page size (per input buffer)	bytes
RS	Average run size	bytes
N	Number of runs ($N \approx \lceil D/RS \rceil$)	count
K	Number of merge operations	count
F	Merge fan-in	count

(M), input size (D), page size (P), and thread counts. We use these constraints to (i) identify when a single merge step is possible and (ii) decide when throttling parallelism is preferable to adding extra merge operations.

Sort–merge pipeline and knobs. We consider the classic two-phase sort–merge pipeline: (1) *run generation* using replacement selection, and (2) a *range-partitioned parallel merge*. Crucially, we allow different parallelism in the two phases: T_{gen} threads for run generation and T_{merge} threads for merging, with an upper bound of $T_{\max}$ available hardware threads.

3.1 Memory constraints

Run Generation (Replacement Selection). Replacement selection produces an average run of size RS using roughly $RS/2$ memory per thread [43]. With T_{gen} independent run generators, respecting the memory budget requires

$$\frac{RS}{2} \cdot T_{\text{gen}} \le M \quad \Longrightarrow \quad RS \le \frac{2M}{T_{\text{gen}}}. \tag{1}$$

Here, we ignored small per-thread overheads and output buffering for clarity.

Single-step Range-Partitioned Merge. Range partitioning assigns each merge thread a key range and has it merge that range from all runs. Each merge thread keeps one input buffer (one page) per run, so the merge fits in memory in a *single-step* if

$$N \cdot T_{\text{merge}} \cdot P \le M. \tag{2}$$

Using $N \approx D/RS$, we obtain a lower bound on run size:

$$\frac{D}{RS} \cdot T_{\text{merge}} \cdot P \le M \quad \Longrightarrow \quad RS \ge \frac{D\, T_{\text{merge}}\, P}{M}. \tag{3}$$

We ignore output buffers during merge; for $N \gg 1$, the $O(T_{\text{merge}}P)$ output footprint is negligible compared to $O(NT_{\text{merge}}P)$ input buffers. Using multiple buffers per run (e.g., double buffering) tightens Eq. 2.

Feasible Window for Single-step Merge. Combining Eq. 1 and Eq. 3, a single-step merge is possible iff there exists an RS satisfying

$$\frac{D\, T_{\text{merge}}\, P}{M} \;\le\; RS \;\le\; \frac{2M}{T_{\text{gen}}}. \tag{4}$$

Equivalently, the window is non-empty iff

$$T_{\text{gen}} \cdot T_{\text{merge}} \;\le\; \frac{2M^2}{D\,P}. \tag{5}$$

Unpublished working draft. Not for distribution.

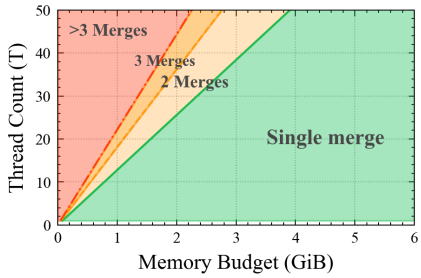


Figure 1: Merge Constraints under $T_{\text{gen}} = T_{\text{merge}} = T$. — The single merge region vanishes once T exceeds the bound in Eq. 6. Calculated for $D = 200$ GiB, $M = 2$ GiB, $P = 64$ KiB.

Eq. 5 exposes the key planning rule: *single-step stability depends on the product of the phase parallelisms.* Increasing parallelism in one phase must be compensated by reducing the other phase.

3.2 Baseline: Symmetric Parallelism

Many external sorts tie the two phases to the same parallelism, $T_{\text{gen}} = T_{\text{merge}} = T$, using available hardware threads. Substituting into Eq. 5 yields the familiar single-step boundary

$$T \le T^* := \frac{M\sqrt{2}}{\sqrt{D\,P}}. \tag{6}$$

When $T > T^*$, the feasible window in Eq. 4 is empty under symmetric parallelism: more threads shrink the per-thread run memory (smaller RS), increasing the run count N and therefore the merge-buffer footprint. Figure 1 illustrates this effect. When $T \le T^*$, single-step merging is feasible (green region); when $T > T^*$, the feasible region vanishes (yellow, orange and red regions).

3.3 When Baseline Exceeds T^*: Multi-step Merging

If $T_{\max} > T^*$ and we insist on symmetric parallelism ($T_{\text{gen}} = T_{\text{merge}} = T_{\max}$), then a single-step merge is infeasible and the merge must be performed in multiple steps. Let K denote the number of merge operations. Under an optimal merge strategy, we iteratively merge the shortest runs first [43, 51]. Each merge operation merges approximately N/K runs. This reduces the per-merge number of runs that must be buffered and relaxes the merge-memory lower bound by a factor of K:

$$RS \ge \frac{D\,T_{\text{merge}}\,P}{K\,M}. \tag{7}$$

This expands the feasible window in Eq. 4 to

$$\frac{D\,T_{\text{merge}}\,P}{K\,M} \le RS \le \frac{2M}{T_{\text{gen}}}. \tag{8}$$

The multi-step feasibility condition is

$$T_{\text{gen}} \cdot T_{\text{merge}} \le \frac{2K\,M^2}{D\,P}. \tag{9}$$

In the symmetric case ($T_{\text{gen}} = T_{\text{merge}} = T$), this becomes

$$T \le \frac{M\sqrt{2K}}{\sqrt{D\,P}}. \tag{10}$$

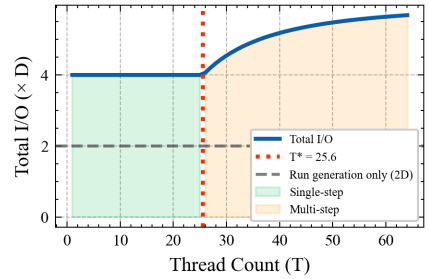


Figure 2: I/O Cost under $T_{\text{gen}} = T_{\text{merge}} = T$. — Total I/O is derived from Huffman-merge analysis. When $T \le T^*$ (Eq. 6), a single step merge is feasible ($L_{\text{avg}} = 1$) and total I/O is $4D$. When $T > T^*$, multi-step merging is required, increasing L_{avg} and total I/O. Calculated for $D = 200$ GiB, $M = 2$ GiB, $P = 64$ KiB.

Figure 1 illustrates the relationship between memory and threads for different K. Increasing K allows for higher symmetric parallelism T at the cost of additional I/O during merging.

3.4 I/O Cost of Multi-step Merging

Run Generation I/O. Run generation reads D bytes and writes D bytes of sorted runs. Assuming pre-formatted binary input, the I/O cost is

$$\text{Run Gen I/O} = 2D. \tag{11}$$

Merge I/O (Huffman-merge). The I/O cost depends on the merge schedule. We use the *Huffman-merge* [38, 43, 51, 59], which minimizes total bytes moved by (i) padding with $d \equiv (1-N) \bmod (F-1)$ zero-size dummies so $N' = N + d \equiv 1 \pmod{F-1}$ and (ii) repeatedly merging the F shortest runs until one run remains.[1] Assume N equal-size runs of RS bytes (total data $D = N\,RS$) and let $h = \lceil \log_F N' \rceil$. In the optimal F-ary merge tree for equal weights, all leaves lie on exactly two depths, $h-1$ and h. If t leaves are at depth $h-1$, then the remaining $N'-t$ are at depth h, and a standard counting argument yields

$$N' = t + F\left(F^{h-1} - t\right) = F^h - (F-1)t \quad \Rightarrow \quad t = \frac{F^h - N'}{F-1}. \tag{12}$$

Since the t shallower positions are assigned to real runs, the average number of merge levels *over real data* is

$$L_{\text{avg}} = \frac{t(h-1) + (N-t)h}{N} = h - \frac{F^h - N'}{(F-1)\,N}. \tag{13}$$

Thus the total merge I/O (reads + writes) is

$$\text{Merge I/O} = 2\,D\,L_{\text{avg}} = 2\,N\,RS\left(h - \frac{F^h - N'}{(F-1)\,N}\right). \tag{14}$$

As a sanity check, for single-step merge ($h = 1$, $N' \le F$), we have $t = \frac{F-N'}{F-1} = 0$ when $N' = F$, yielding $L_{\text{avg}} = 1$ and merge I/O of $2D$, which correctly accounts for one read and one write of the entire dataset. Therefore, the total I/O cost of the sort is

$$\text{Total I/O} = 2D + 2D\,L_{\text{avg}} = 2D(1 + L_{\text{avg}}). \tag{15}$$

[1] Dummies shape the tree but contribute no I/O.

Unpublished working draft. Not for distribution.

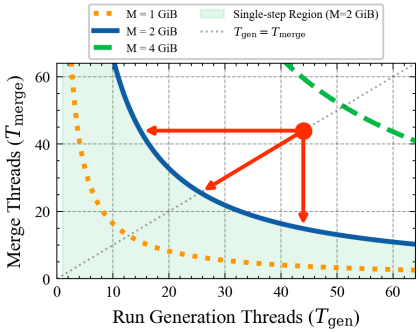


Figure 3: Single-step Feasibility in Thread Space. — The boundary $T_{gen}T_{merge} = 2M^2/(DP)$ (Eq. 5) separates configurations that admit a single-step merge (shaded, below the curve) from those that require multi-step merging. The baseline point (T_{max}, T_{max}) may lie above the boundary; restoring single-step feasibility requires throttling run generation (move left), throttling merge (move down), or splitting throttling across both phases (move diagonally). Shown for $D = 200$ GiB, $M = 2$ GiB, $P = 64$ KiB

This model quantifies the I/O penalty of multi-step merging (larger L_{avg}) relative to single-step merging. The total I/O cost with respect to the thread count is illustrated in Figure 2.

3.5 Avoiding Multi-step: Decoupling and Throttling

Multi-step merging is not the only way to restore feasibility. When the baseline violates Eq. 6, we can instead decouple the phase parallelism and choose (T_{gen}, T_{merge}) to satisfy the general single-step condition in Eq. 5:

$$T_{gen}T_{merge} \leq \frac{2M^2}{DP}. \tag{16}$$

Two natural extremes are: (i) throttle run generation ($T_{gen} < T_{max}$, $T_{merge} = T_{max}$), or (ii) throttle merge ($T_{gen} = T_{max}$, $T_{merge} < T_{max}$). In both cases, we recover a non-empty run-size window (Eq. 4) and complete the sort with a single merge step. Figure 3 illustrates the trade-off between T_{gen} and T_{merge}. The hyperbolic boundary defined by Eq. 5 separates the feasible single-step region (shaded) from the multi-step regime. When full parallelism is infeasible, we can move back into the stable zone by (1) throttling run generation (moving left), (2) throttling merge (moving down), or (3) balancing both phases.

4 Implementation Details

We implement a disk-to-disk external sorting engine in Rust, designed for sorting large datasets that exceed available memory. The engine currently supports row format inputs that support parallel scanning. It writes out sorted data in a custom binary format that writes out sort keys and payloads sequentially with their lengths, so that downstream operators can read them efficiently. The implementation follows the structure of classical parallel external merge sort [72], with a planner that allocates memory and threads based on available RAM and CPU cores. The algorithm proceeds in three phases: (1) *parallel run generation*, (2) *quantile–based boundary selection*, and (3) *parallel range–restricted merging*. The goal is to exploit resources no more than necessary without lowering throughput. The prior section derived practical rules for configuring memory budgeting and parallelism. The following subsections assume that these parameters are set according to those rules.

4.1 Parallel Run Generation

Given a number of threads T, we first partition the input file into T disjoint sets by the size of the file. Each thread is assigned one set and processes it independently. Each thread maintains a tournament tree and fills it by reading rows from its assigned set. We use tree-of-losers for the tournament tree. We use replacement selection to form runs: once the tree fills, the thread will start emitting records from the top of the run. A new record from the input will be pushed into the tournament tree if the record is larger than the previously emitted record. Otherwise, it will be buffered to a vector, used to build the next run. This process repeats until all assigned pages of the input file are processed. Each run will keep a few samples from the sorted data to create a sparse index for the subsequent merge phase. Currently, we sample one key every 1000 tuples. We assume that this sparse index is small enough to fit in memory as it is approximately 0.1% of the whole dataset. For each thread, we create a local quantile sketch to summarize the distribution of its assigned data. We implemented two sketching approaches: weighted reservoir sampling [22] and KLL [39, 41]. Keys sampled from each run at some interval are inserted into the sketch. Currently, we set the interval to 100 tuples and set the sketch size to 200. We assume that these sketches occupy a small portion of the memory budget and are negligible. For writing out the files, we use at most one file per thread to avoid allocating too many file descriptors, as most OSes have a limit on the number of open file descriptors per process. Figure 4a illustrates the process.

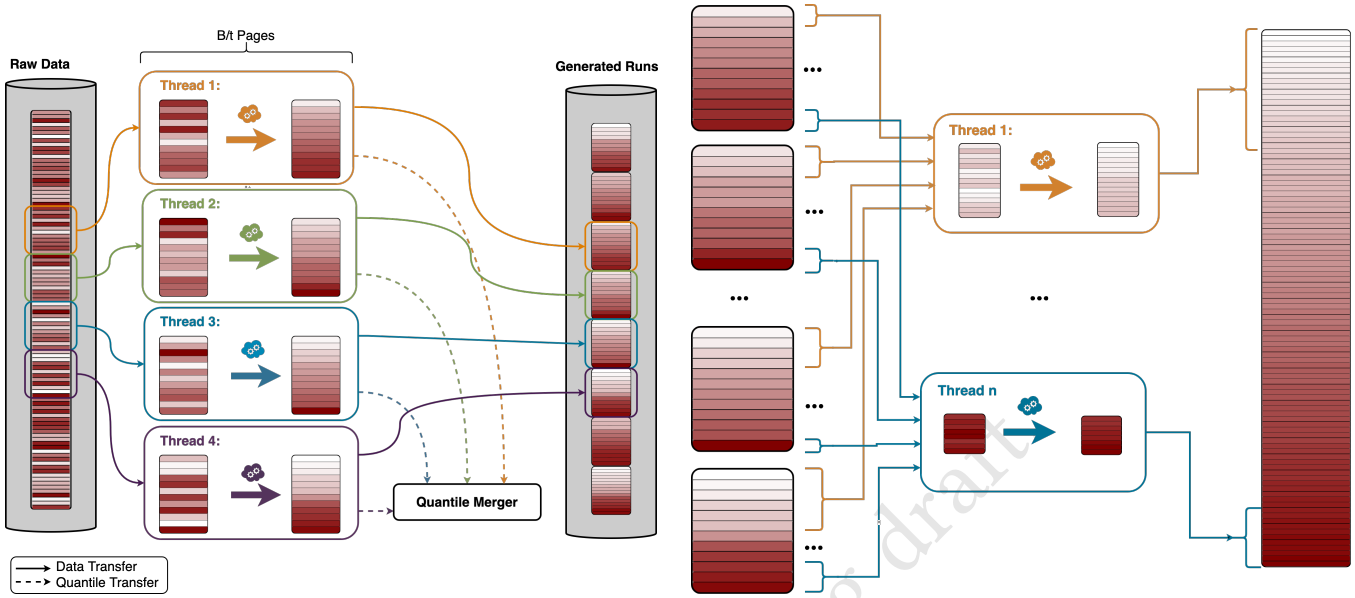
4.2 Quantile-Based Boundary Selection

After run generation, we have T local quantile sketches, one per thread. We merge these sketches into a global sketch and extract $T-1$ quantiles to serve as global boundaries for range partitioning. The goal is to create T partitions with a roughly equal number of tuples, thereby minimizing the "straggler" effect during the subsequent parallel merge phase. We implemented two sketching approaches: **Weighted Reservoir Sampling (WRS)** and the **KLL Sketch**.

- **Weighted Reservoir Sampling:** We utilize the A-Exp algorithm [22], assigning each tuple a uniform weight of 1.0. Local sketches are merged by comparing exponential priorities ($\frac{-\ln(u_i)}{w_i}$) and retaining the k samples with the smallest priority values.
- **KLL Sketch:** We implement the KLL algorithm, which maintains a hierarchy of compactors. Merging is performed by combining buffers at each level and reducing them via randomized downsampling [41].

The primary theoretical difference lies in the maximum partition error. For weighted reservoir sampling with uniform weights (i.e., a uniform sample of size k), standard concentration for empirical quantiles yield rank error $\varepsilon = \tilde{O}(1/\sqrt{k})$ when we require all $T-1$ boundaries to be simultaneously accurate (e.g., via a union bound).

Unpublished working draft. Not for distribution.



(a) Parallel run generation with striped page assignment: thread j processes pages $j, j+t, j+2t, \ldots$, each maintaining a private buffer of $\lfloor B/t \rfloor$ pages.TODO update image

(b) Parallel range-restricted merge: global boundaries split each run into segments; each worker merges only its range. Poor boundaries overload one worker and dominate runtime.

Figure 4: Parallel external sorting pipeline. (a) Run generation; (b) range-restricted merge.

In contrast, KLL provides a distribution-agnostic high-probability rank-error guarantee with $\varepsilon = \tilde{O}(1/k)$ for the same sketch size, and this guarantee is preserved under merges [39, 41].

This rank error directly controls the largest partition size. Let $\bar{P} = N/T$ be the average partition size. If each of the $T-1$ boundaries has rank error at most εN, then each boundary can shift by at most εN tuples, and any partition (bounded by two adjacent boundaries) can gain at most $2\varepsilon N$ tuples. Hence,

$$\max_i |P_i| \;\le\; \frac{N}{T} + 2\varepsilon N \quad\Longrightarrow\quad \frac{\max_i |P_i|}{N/T} \;\le\; 1 + 2\varepsilon T. \tag{17}$$

Substituting the above ε rates, WRS yields $\max_i |P_i|/(N/T) \le 1 + \tilde{O}(T/\sqrt{k})$, whereas KLL yields $\max_i |P_i|/(N/T) \le 1 + \tilde{O}(T/k)$. As shown in our experiments, this tighter balancing is most critical when T is small and a single oversized partition can dominate execution time; when T is large, the relative impact diminishes as partitions become smaller.

4.3 Parallel Range-Restricted Merging

Given the $T-1$ global boundaries, we partition the key space into T ranges and assign each range to a thread. Each thread merges tuples in its assigned range from all runs. Each run is exposed via a range iterator that holds one read page, replaces it with the next page when consumed, and yields only tuples inside the assigned range. The range scan operation will start reading the run from the position corresponding to the lower bound of the range by exploiting the sparse index created in the run generation phase. Each thread maintains a tree-of-losers over active iterators to perform the merge. Each thread also maintains an output buffer of one page to write out the merged result. If the number of active iterators exceeds the maximum fan-in that can be supported by the memory budget, we perform hierarchical merging by iteratively merging the shortest runs in the run pool [43, 51]. Figure 4b illustrates the process.

4.4 Offset Value Codes

The above engine implements OVC to reduce I/O volume and comparison cost. The run generation phase produces OVC as a byproduct of sorting the buffer in memory using tree-of-losers. When writing the run to disk, we write out the OVC along with the sorted data. Additionally, we truncate the keys to remove common prefixes before writing them out. The length of the common prefix is stored as part of the OVC. When reading the run from disk for subsequent merging, we reconstruct the original keys. The sparse index created in the run generation contains actual keys instead of the truncated keys. Thus, starting from the entry in the sparse index, we can reconstruct the original keys from the truncated keys. The output of the merge phase is also encoded using OVC. The run generation and merge phases are modified to support OVC, while the quantile-based boundary selection phase remains unchanged. We use 64-bit OVC which encodes the index of the first different byte in the first 16 bits and the remaining 48 bits are used to store the bytes starting from the first different byte. In order to understand the effectiveness of OVC, we also implement a baseline engine without OVC for comparison.

5 Evaluation

All experiments were conducted on a server equipped with dual-socket Intel® Xeon® Silver 4116 CPUs. Each socket features 12

Unpublished working draft. Not for distribution.

physical cores operating at a clock speed of 2.10 GHz, with hyper-threading enabled (i.e., two threads per core) and 192 GB of RAM. The external storage is an Intel 760p NVMe SSD rated at 340 kIOPS on random-read and 275 kIOPS on random-write for 4 KiB blocks. We employ the Direct I/O interface for all disk operations, ensuring that the pages are not cached in the operating system's page cache. We use offset-value coding (OVC) by default during both run generation and merging to reduce I/O volume. This evaluation seeks to address the following research questions:

(1) **Parallelism.** How should parallelism be applied in run generation and merging to maximize efficiency? Is it beneficial to use asymmetric number of threads for both phases?
(2) **Memory Usage.** What is the optimal memory usage for run generation and merging?
(3) **Offset-Value Coding.** How does the use of offset-value coding affect the end-to-end sorting time?
(4) **Quantile Estimation.** How does the quality of quantile estimation affect the end-to-end sorting time?

We use two datasets for our evaluation, which is summarized in table 2. The first dataset is the GENSORT dataset generated by the `gensort` tool [?], which creates synthetic key-value pairs. A key is a 10-byte binary string, and the value is a 90-byte binary string. The key-value pairs are generated in a random order and stored row-wise in a binary file. We generated 200 GiB of data for this evaluation. The second dataset is the TPC-H benchmark's `lineitem` table [2], which is widely used for evaluating analytical database systems. The `lineitem` table contains various data types, including integers, floating-point numbers, dates, and strings. We sort by string columns: `l_returnflag`, `l_linestatus`, `l_shipinstruct`, `l_shipmode`, and `l_comment`, with cardinalities of 3, 2, 4, 7, and nearly unique values, respectively. We chose a scale factor of 500 for this evaluation, resulting in an input CSV file size of approximately 380 GiB. After projecting the necessary columns for sorting (i.e., the total size of all the normalized keys and payload bytes), the data size was approximately 200 GiB.

The two datasets provide different characteristics for evaluating sorting performance. The GENSORT dataset has fixed-length keys and values, allowing us to assess the performance of sorting algorithms on uniform data. In contrast, the TPC-H `lineitem` table contains variable-length string columns, which introduce additional complexity due to varying key sizes and memory indirection. The keys and the payloads are extracted from the table and stored in a row-wise binary format before sorting along with a sparse index which maps key positions to their offsets in the binary file (per 4 MiB chunk).

5.1 Parallelism and Phase Decoupling

Symmetric Configuration (The Baseline). We first evaluate the impact of symmetric parallelism, where the thread count T is identical for both run generation and merging ($T_{gen} = T_{merge} = T$). We fixed the memory budget to 2 GiB and varied T from 4 to 44. Figures 5 and 6 present the sorting latency, total I/O volume, and effective I/O bandwidth for GENSORT and TPC-H `lineitem`, respectively.

We observe distinct scaling behaviors driven by data entropy. For GENSORT (high entropy), performance improves with parallelism

Table 2: Dataset Specifications and Storage Characteristics

Dataset	Key and Payload	Characteristics
GENSORT [?]	**Sort Key:** 10-byte **Payload:** 90-byte *Dist:* Uniform random	Fixed-length records; Row-wise binary storage; Synthetic workload; Relatively larger payload compared with the sort key.

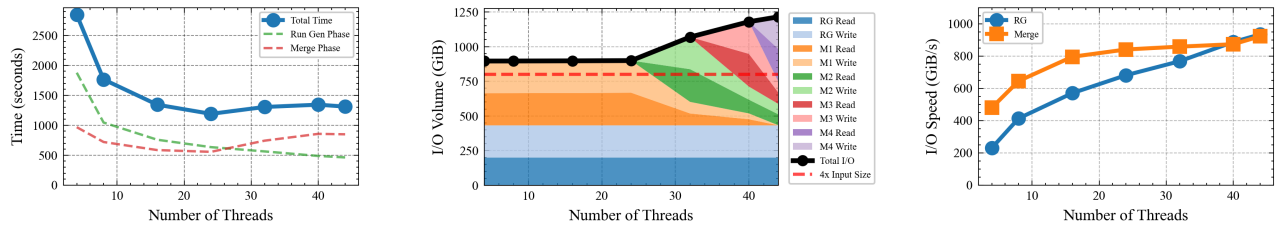
TPC-H [2] `lineitem`	**Sort Key (Card.):**
	`l_returnflag` (3)
	`l_linestatus` (2)
	`l_shipinstruct` (4)
	`l_shipmode` (7)
	`l_comment` (Unique)
	Payload:
	`l_orderkey`
	`l_linenumber`
		Variable-length keys; High memory indirection overhead; Pre-converted to row-wise binary w/ sparse index (4 MiB chunks); Relatively smaller payload compared with sort key.

only up to a saturation point of $T \approx 24$. Beyond this threshold, latency degrades. This inversion is caused by I/O contention: as T increases, the per-thread memory buffer shrinks, forcing the system into multi-pass merging which amplifies I/O volume (Figure 5b) and saturates the SSD bandwidth (Figure 5c). In contrast, `lineitem` (low entropy) exhibits a "compression cushion." The use of Offset Value Codes (OVC) significantly reduces the write volume by truncating shared key prefixes . This increases the effective I/O bandwidth, allowing the system to sustain performance even at high thread counts where symmetric over-provisioning would typically cause thrashing. Thus, while GENSORT hits a hard "physical wall," `lineitem` degrades gracefully.

Asymmetric Configuration (The Decoupling Strategy). Next, we evaluate asymmetric configurations to test the hypothesis that decoupling the phases improves resource efficiency. We independently varied T_{gen} and T_{merge} to generate the throughput and cost-efficiency heatmaps shown in Figure 7. Here, cost is defined as *occupancy cost* (thread-seconds), capturing both computational work and I/O wait times. The results reveal a structural divergence in resource requirements:

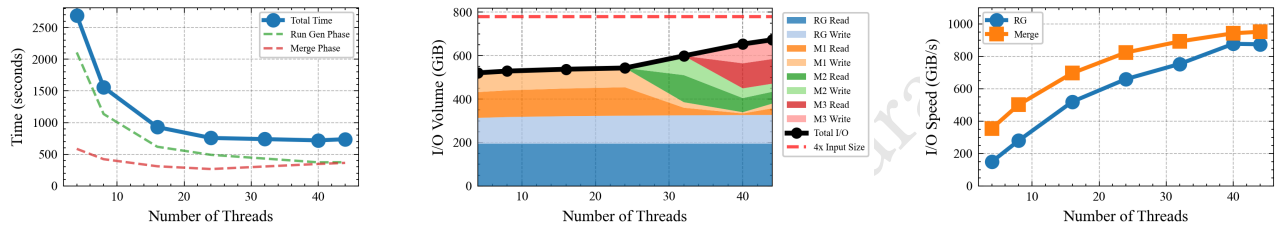
(1) **Throughput Maximization:** across both datasets, peak throughput is achieved when $T_{gen} > T_{merge}$. This confirms that run generation is relatively CPU-bound (benefiting from thread scaling to hide sorting latency), while merging is I/O-bound (requiring thread throttling to prevent disk saturation).
(2) **The Efficiency Gap:** For GENSORT, the optimal operating zone is narrow. Because the merge phase lacks OVC compression, the I/O channel saturates early. Consequently, adding merge threads beyond the single-pass threshold ($T_{merge} \approx 24$) degrades efficiency without improving speed.
(3) **Robustness via OVC:** For `lineitem`, the heatmap reveals a broad "robust zone." The compression afforded by OVC lowers the merge cost relative to run generation. This allows the planner to be more aggressive with run generation threads (T_{gen}) to maximize CPU utilization, while being more lenient with merge threads (T_{merge}), as the reduced I/O volume minimizes the penalty of over-provisioning.

Unpublished working draft. Not for distribution.



(a) Sorting Time **(b) I/O Analysis** **(c) I/O Speed Analysis**

Figure 5: External Sort of GenSort with Varying Thread Count. $T_{gen} = T_{merge}$ — Panels: (a) sorting time, (b) I/O analysis, (c) I/O speed analysis. Input data size ≈ 200 GiB; memory budget 2 GiB; threads varied from 4 to 44.



(a) Sorting Time **(b) I/O Analysis** **(c) I/O Speed Analysis**

Figure 6: External Sort of TPC-H `lineitem` (SF500) with Varying Thread Count. — Panels: (a) sorting time, (b) I/O analysis, (c) I/O speed analysis. Input data (keys + payload) ≈ 200 GiB; memory budget 2 GiB; threads varied from 4 to 44.

In summary, asymmetric decoupling allows us to harvest the latency benefits of high parallelism in the CPU-bound phase without paying the contention tax in the I/O-bound phase.

5.2 Memory Size for Run Generation and Merging

In the previous experiment, we fixed the total memory usage to 2 GiB. In this experiment, we want to see the effect of having more memory available. When more memory is available, we can create longer runs during run generation, which reduces the number of runs and the depth of merging. We fixed the number of threads to 40 and varied the memory size from 1 GiB to 32 GiB. The sort buffer size for run generation was set to available memory divided by the number of threads.

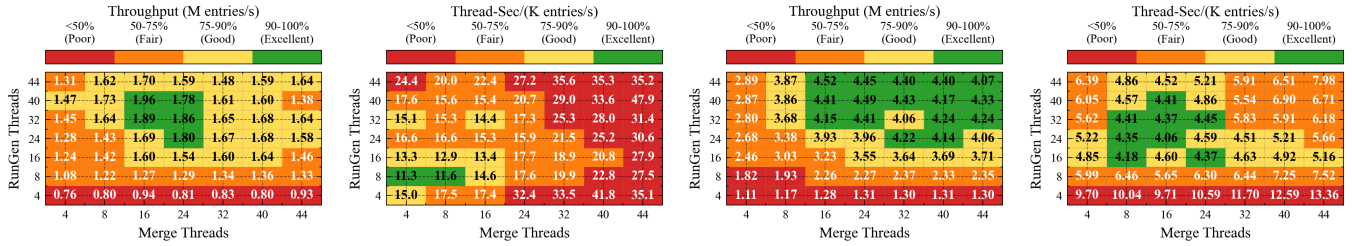
Figure 8 shows the sorting time with varying memory size. For both datasets, the sorting time decreases as the memory size increases up to a certain point. This is because with smaller memory, we have to create shorter runs, which increases the number of merge steps. This results in increased I/O volume, which increases the total sorting time. However, when the memory size exceeds a certain threshold (around 4 GiB), the sorting time starts to increase. This is because with more memory, the run generation phase becomes slower due to increased cache misses. The `lineitem` dataset has high memory indirections during run generation due to longer keys with shared prefixes, which exacerbates cache misses during run generation. Therefore, increasing the memory size beyond a certain point does not improve the sorting time.

5.3 OVC Impact on End-to-End Sorting

In the previous experiments, we have fully utilized offset-value coding (OVC) during both run generation and merging. Specifically, we use replacement selection using tree-of-losers enhanced with OVC during run generation and merge. The run generation will output a run that truncates the shared prefix based on the previous entry of the run. In this experiment, we want to see how OVC affects the end-to-end sorting time by disabiling OVCs during the sort. That is, we use a regular replacement selection and merge using tree-of-losers without OVC support.

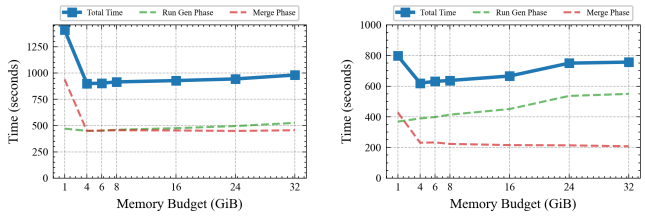
Figure 9 shows the impact of OVC on the end-to-end sorting time for the GenSort and TPC-H `lineitem` datasets. For this experiment, we set the total memory size to 2GiB and varied the thread count. The results show that OVC does not have a significant impact on the end-to-end sorting time for GenSort, whereas it has a substantial impact on TPC-H `lineitem`, especially when the number of threads is large. The reason for the difference lies in the entropy of the key prefixes and how that interacts with OVC. For GenSort, because the keys are random bytes, the probability that two keys share the same first 6 bytes is low. This means opportunities for compression using OVC is low. However, for TPC-H `lineitem`, the keys have long shared prefixes, which means that OVC can significantly help reduce the I/O volume. When the number of merge steps increases with the number of threads, the benefit of OVC becomes more pronounced. All in all, OVC can significantly improve the end-to-end sorting time for keys with long shared prefixes.

Unpublished working draft. Not for distribution.



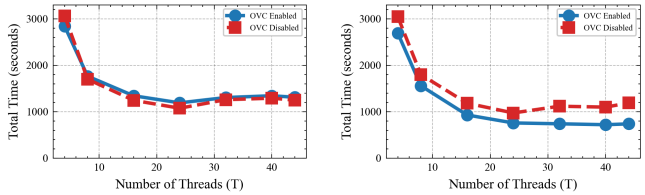
(a) GenSort: Throughput (b) GenSort: Cost per Throughput (c) lineitem: Throughput (d) lineitem: Cost per Throughput

Figure 7: Asymmetric Thread Allocation Analysis. — Heatmaps showing (a–b) GenSort and (c–d) `lineitem` performance with varying thread counts for run generation (y-axis) and merge (x-axis). Panels show: (a,c) throughput; (b,d) cost per throughput. Color coding represents efficiency: $\eta = T/T_{\max}$ for throughput and $\eta = C_{\min}/C$ for cost, with tiers at 50%, 75%, and 90%.



(a) GenSort (b) TPC-H `lineitem`

Figure 8: Memory Cliff Effect in External Sort. — Sorting time with varying memory size. Input data size ≈ 200 GiB; 40 threads; memory varied from 1 GiB to 32 GiB.



(a) GenSort (b) TPC-H `lineitem`

Figure 9: Impact of OVC on Total Sorting Time. — Comparison of OVC vs non-OVC configurations for (a) GenSort and (b) TPC-H `lineitem` datasets.

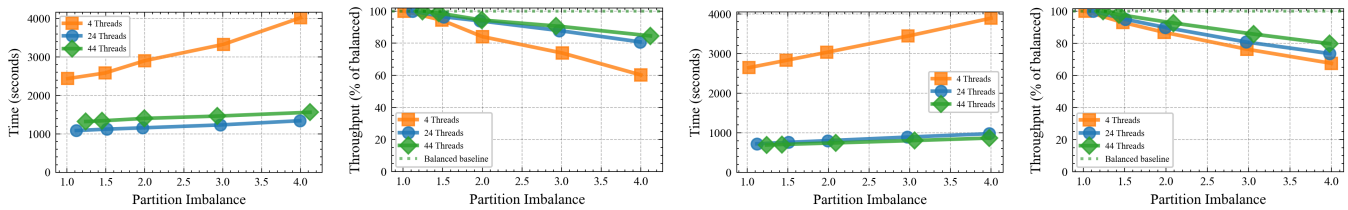
5.4 Quantile Estimation

Previous experiments used KLL sketches to estimate quantiles for partitioning merge work among threads. In this experiment, we want to see how different quantile estimation techniques affect the end-to-end sorting time. We compared two quantile estimation techniques: (1) weighted reservoir sampling and (2) KLL sketches. First we measured, how imbalance in partition sizes affect the sorting time by injecting artificial imbalance for merging partitions. Then, we compared the two quantile estimation techniques based on the end-to-end sorting time.

Impact of Imbalance. We first measured how imbalance in partition sizes affect the sorting time by injecting artificial imbalance for merging partitions. We fixed the memory size to 2 GiB and the number of threads to 4, 24, 44. We modified the KLL sketch algorithm to produce different levels of imbalance from 1.0 (perfectly balanced) to 4.0 (one partition is four times as large as the average partition size). The actual imbalance is not exactly the same as the target imbalance due to the randomness in quantile estimation. Figure 10 shows the sorting time and throughput degradation with varying imbalance for the GenSort and TPC-H `lineitem` datasets, respectively. As expected, as the imbalance increases, the sorting time (specifically the merge time) increases due to the straggler effect. However, the impact of the imbalance varies with the number of threads. When the number of threads is small, the impact of imbalance is more pronounced because each thread has to process a larger partition, and the straggler effect becomes more significant. However, when the number of threads is large, the impact of imbalance is less pronounced because each thread has a smaller partition to process, and the straggler effect is mitigated.

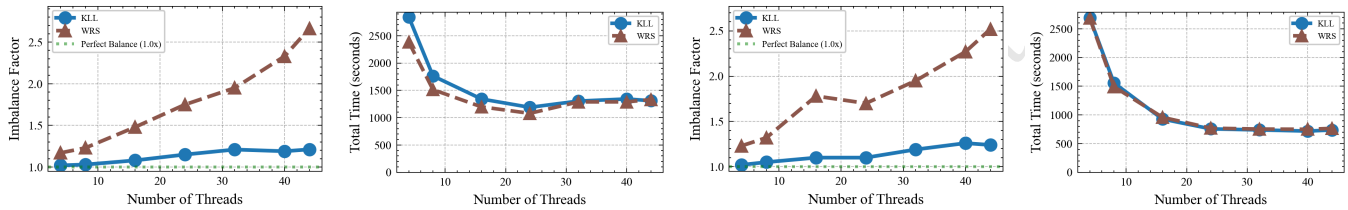
Weighted Reservoir Sampling vs KLL. Next, we compared different quantile estimation techniques: (1) weighted reservoir sampling and (2) KLL sketches. The weighed reservoir sampling associates randomized weights to each sampled entry and keeps the top-k entries based on the weights. This technique is simple but has worse rank error guarantees compared to KLL sketches as discussed in Section ??. We varied the number of threads from 4 to 44 and fixed the memory size to 2 GiB. Figure 11 shows the end-to-end sorting time and imbalance for the two quantile estimation techniques for the GenSort and TPC-H `lineitem` datasets, respectively. The results show that KLL sketches consistently outperform weighted reservoir sampling in terms of imbalance. However, the end-to-end sorting time is not significantly different between the two techniques. This is because the when the number of threads is small, the imbalance is not significant enough to affect the overall sorting time. However, as the number of threads increases, although KLL sketches produce better imbalance, the overall sorting time improvement is marginal because each thread has a smaller partition to process, and the straggler effect is mitigated. In summary, the choice of quantile estimation techniques do not significantly affect the end-to-end sorting time in our experiments. However, for robust performance across different configurations, KLL sketches are preferred due to their better rank error guarantees.

Unpublished working draft. Not for distribution.



(a) GenSort: Time vs Imbalance (b) GenSort: Throughput (c) `lineitem`: Time vs Imbalance (d) `lineitem`: Throughput

Figure 10: Impact of Partition Imbalance on Sorting Performance. — Panels: (a–b) GenSort and (c–d) `lineitem` showing time vs imbalance and throughput degradation with varying imbalance. Imbalance is measured as max/avg partition size.



(a) GenSort: Imbalance (b) GenSort: Time (c) `lineitem`: Imbalance (d) `lineitem`: Time

Figure 11: Quantile Estimation Method Comparison. — Panels: (a–b) GenSort and (c–d) `lineitem` showing imbalance and total time comparison for different quantile estimation techniques. Imbalance is measured as max/avg partition size.

Unpublished working draft. Not for distribution.

6 Lessons Learned

Our evaluation of resource-aware external sorting highlights three critical insights regarding resource usage, data entropy, and algorithmic robustness:

- **Decoupling Parallelism via Asymmetry:** The distinct physical characteristics of the sorting phases necessitate independent resource planning. Run generation is fundamentally CPU-bound and benefits from adding more threads. In contrast, the merge phase is typically I/O-bound; providing symmetric parallelism often leads to resource contention and diminishing returns. By adopting an *asymmetric configuration*—saturating the CPU during generation while throttling concurrency during merging—the system achieves a Pareto-optimal balance, maximizing throughput while minimizing resource waste.
- **Entropy Dictates Operational Robustness:** The impact of Offset Value Codes (OVC) extends beyond comparison efficiency; it fundamentally alters the effective I/O bandwidth. For low-entropy workloads (e.g., TPC-H Lineitem), OVC enables significant compression, effectively multiplying the I/O throughput and creating a "robustness cushion" that tolerates higher merge parallelism. Conversely, high-entropy workloads (e.g., GenSort) lack this compression benefit, exposing the raw I/O bottleneck. This makes the system highly sensitive to over-provisioning, reinforcing the need for precise, entropy-aware configuration.
- **Robustness of Quantile Estimation:** While precise partitioning is theoretically critical for parallel sorting, our experiments reveal that the system is relatively insensitive to minor quantile approximation errors. At low parallelism, coarse-grained splitting is sufficient to balance large partitions. At high parallelism, while the risk of skew increases, the reduced granularity of per-thread tasks diminishes the absolute impact of stragglers. Nevertheless, we find that KLL sketches are more robust than priority-based reservoir sampling. KLL provides deterministic rank error guarantees rather than probabilistic variance, offering a safety net that ensures load balance stability across worst-case data distributions.

7 Conclusion

In this work, we presented a comprehensive study of resource-aware external sorting, introducing practical strategies for configuring memory and CPU usage across the distinct phases of run generation and merging. By leveraging Offset Value Codes (OVC), we demonstrated significant reductions in I/O volume and comparison costs, particularly for workloads with low-entropy sort keys. Our quantile-based partitioning approach, utilizing both Weighted Reservoir Sampling and KLL sketches, enabled efficient parallel merging with balanced workload distribution and minimal synchronization overhead. Our experimental evaluation across diverse datasets highlighted the effectiveness of our methods, revealing critical insights into the interplay between resource allocation, data entropy, and algorithmic robustness. Future work may explore adaptive strategies that dynamically adjust resource allocation based on real-time workload characteristics, further enhancing the efficiency and scalability of external sorting in modern data processing environments.

References

[1] [n. d.]. Sort(1) - Linux Manual Page. https://man7.org/linux/man-pages/man1/sort.1.html

[2] 1999. TPC-H Homepage. https://www.tpc.org/tpch/

[3] Khaled Alsabti, Sanjay Ranka, and Vineet Singh. 1997. A One-Pass Algorithm for Accurately Estimating Quantiles for Disk-Resident Data. In *Proceedings of the 23rd International Conference on Very Large Data Bases*. 346–355.

[4] Nancy Amato, Ravishankar Iyer, Sharad Sundaresan, and Yan Wu. 1998. *A Comparison of Parallel Sorting Algorithms on Different Architectures*. Technical Report. Texas A & M University.

[5] Lars Arge, Octavian Procopiuc, Sridhar Ramaswamy, Torsten Suel, and Jeffrey Scott Vitter. 1998. Scalable Sweeping-Based Spatial Join. In *Proceedings of the 24rd International Conference on Very Large Data Bases*. 570–581.

[6] Jean-Loup Baer and Yi-Bing Lin. 1989. Improving Quicksort Performance with a Codeword Data Structure. *IEEE Transactions on Software Engineering* 15, 5 (1989), 622–631.

[7] R. Bayer and E. McCreight. 1970. Organization and Maintenance of Large Ordered Indices. In *Proceedings of the 1970 ACM SIGFIDET (Now SIGMOD) Workshop on Data Description, Access and Control*. 107–141.

[8] Dina Bitton and David J. DeWitt. 1983. Duplicate Record Elimination in Large Data Files. *ACM Transactions on Database Systems* 8, 2 (June 1983), 255–265.

[9] Guy E. Blelloch, Charles E. Leiserson, Bruce M. Maggs, C. Greg Plaxton, Stephen J. Smith, and Marco Zagha. 1991. A Comparison of Sorting Algorithms for the Connection Machine CM-2. In *Proceedings of the Third Annual ACM Symposium on Parallel Algorithms and Architectures - SPAA '91* (Hilton Head, South Carolina, United States, 1991). ACM Press, 3–16. doi:10.1145/113379.113380

[10] Peter Boncz, Yannis Chronis, Jan Finis, Stefan Halfpap, Viktor Leis, Thomas Neumann, Anisoara Nica, Caetano Sauer, Knut Stolze, and Marcin Zukowski. 2023. SPA: Economical and Workload-Driven Indexing for Data Analytics in the Cloud. In *ICDE*. 3740–3746.

[11] Panagiotis Bouros, Nikos Mamoulis, Dimitrios Tsitsigkos, and Manolis Terrovitis. 2021. In-Memory Interval Joins. *The VLDB Journal* 30, 4 (July 2021), 667–691.

[12] Yannis Chronis, Thanh Do, Goetz Graefe, and Keith Peters. 2020. External Merge Sort for Top-k Queries: Eager Input Filtering Guided by Histograms. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 2423–2437.

[13] Graham Cormode, Abhinav Mishra, Joseph Ross, and Pavel Veselý. 2021. Theory meets practice at the median: A worst case comparison of relative error quantile algorithms. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 2722–2731.

[14] Carlo Curino, Eugene Wu, Evan P C Jones, Sam Madden, Raluca Ada Popa, Hari Balakrishnan, Nirmesh Malviya, and Nickolai Zeldovich. 2011. Relational Cloud: A Database-as-a-Service for the Cloud. In *CIDR*.

[15] Sudipto Das, Shoji Nishimura, Divyakant Agrawal, and Amr El Abbadi. 2010. Live Database Migration for Elasticity in a Multitenant Database for Cloud Platforms. *UCSB Computer Science Technical Report* 9 (2010).

[16] Jochen Van den Bercken and Bernhard Seeger. 2001. An Evaluation of Generic Bulk Loading Techniques. In *Proceedings of the 27th International Conference on Very Large Data Bases*. 461–470.

[17] D.J. DeWitt, J.F. Naughton, and D.A. Schneider. 1991. Parallel Sorting on a Shared-Nothing Architecture Using Probabilistic Splitting. In *[1991] Proceedings of the First International Conference on Parallel and Distributed Information Systems* (1991-02). 280–291. doi:10.1109/PDIS.1991.183115

[18] David J. DeWitt, Jeffrey F. Naughton, and Donovan A. Schneider. 1991. Parallel Sorting on a Shared-Nothing Architecture Using Probabilistic Splitting. In *Proceedings of the First International Conference on Parallel and Distributed Information Systems*. 280–291.

[19] Thanh Do and Goetz Graefe. 2023. Robust and Efficient Sorting with Offset-Value Coding. *ACM Transactions on Database Systems* 48, 1, Article 2 (March 2023).

[20] Thanh Do, Goetz Graefe, and Jeffrey Naughton. 2023. Efficient Sorting, Duplicate Removal, Grouping, and Aggregation. *ACM Transactions on Database Systems* 47, 4, Article 16 (Jan. 2023).

[21] Bin Dong, Surendra Byna, and Kesheng Wu. 2016. SDS-sort: Scalable Dynamic Skew-Aware Parallel Sorting. In *Proceedings of the 25th ACM International Symposium on High-Performance Parallel and Distributed Computing*. 57–68.

[22] Pavlos S. Efraimidis and Paul G. Spirakis. 2006. Weighted Random Sampling with a Reservoir. *Inform. Process. Lett.* 97, 5 (2006), 181–185.

[23] Aaron J. Elmore, Sudipto Das, Divyakant Agrawal, and Amr El Abbadi. 2011. Zephyr: Live Migration in Shared Nothing Databases for Elastic Cloud Platforms. In *SIGMOD*. 301–312.

[24] Jost Enderle, Matthias Hampel, and Thomas Seidl. 2004. Joining Interval Data in Relational Databases. In *Proceedings of the 2004 ACM SIGMOD International Conference on Management of Data*. 683–694.

[25] Dina Bitton Friedland. 1981. *Design, Analysis, and Implementation of Parallel External Sorting Algorithms*. Ph. D. Dissertation. The University of Wisconsin - Madison.

[26] Dengfeng Gao, S. Jensen, T. Snodgrass, and D. Soo. 2005. Join Operations in Temporal Databases. *The VLDB Journal* 14, 1 (March 2005), 2–29.

[27] Goetz Graefe. 1993. Query Evaluation Techniques for Large Databases. *Acm Computing Surveys* 25, 2 (June 1993), 73–169.

[28] Goetz Graefe. 2003. Sorting And Indexing With Partitioned B-Trees. In *CIDR*.

[29] Goetz Graefe. 2006. Implementing Sorting in Database Systems. *ACM Comput. Surv.* 38, 3 (Sept. 2006), 10–es.

[30] Goetz Graefe. 2008. A General and Efficient Algorithm for "Top" Queries. In *2008 IEEE 24th International Conference on Data Engineering Workshop*. 548–555.

[31] Goetz Graefe. 2011. Modern B-Tree Techniques. *Found. Trends Databases* 3, 4 (April 2011), 203–402.

[32] Goetz Graefe. 2024. More Modern B-Tree Techniques. *Found. Trends Databases* 13, 3 (2024), 169–249.

[33] Goetz Graefe and Thanh Do. 2022. Offset-Value Coding in Database Query Processing. In *International Conference on Extending Database Technology*. https://api.semanticscholar.org/CorpusID:252683191

[34] Goetz Graefe, Marius Kuhrt, and Bernhard Seeger. 2025. Modifying an Existing Sort Order with Offset-Value Codes. In *Proceedings 28th International Conference on Extending Database Technology, EDBT 2025, Barcelona, Spain, March 25-28, 2025*. 246–254.

[35] G. Graefe, A. Linville, and L. D. Shapiro. 1994. Sort vs. Hash Revisited. *IEEE Trans. on Knowl. and Data Eng.* 6, 6 (Dec. 1994), 934–944.

[36] Michael Greenwald and Sanjeev Khanna. 2001. Space-efficient online computation of quantile summaries. *ACM SIGMOD Record* 30, 2 (2001), 58–66.

[37] Xiao Hu, Stavros Sintos, Junyang Gao, Pankaj K. Agarwal, and Jun Yang. 2022. Computing Complex Temporal Join Queries Efficiently. In *Proceedings of the 2022 International Conference on Management of Data*. 2076–2090.

[38] David A. Huffman. 1952. A Method for the Construction of Minimum-Redundancy Codes. *Proceedings of the IRE* 40, 9 (1952), 1098–1101.

[39] Nikita Ivkin, Edo Liberty, Kevin J. Lang, Zohar S. Karnin, and Vladimir Braverman. 2022. Streaming Quantiles Algorithms with Small Space and Update Time. *Sensors* 22, 24 (2022), 9612.

[40] B. Iyer and D.M. Dias. 1990. System Issues in Parallel Sorting for Database Systems. In *[1990] Proceedings. Sixth International Conference on Data Engineering*. 246–255.

[41] Zohar Karnin, Kevin Lang, and Edo Liberty. 2016. Optimal Quantile Approximation in Streams. In *2016 IEEE 57th Annual Symposium on Foundations of Computer Science (FOCS)*. 71–78.

[42] Zohar Karnin, Kevin Lang, and Edo Liberty. 2016. Optimal quantile approximation in streams. In *2016 ieee 57th annual symposium on foundations of computer science (focs)*. IEEE, 71–78.

[43] Donald E. Knuth. 1998. *The Art of Computer Programming, Volume 3: (2nd Ed.) Sorting and Searching*.

[44] Ani Kristo and Tim Kraska. 2023. *Parallel External Sorting of ASCII Records Using Learned Models*. doi:10.48550/arXiv.2305.05671 arXiv:2305.05671 [cs]

[45] Ani Kristo and Tim Kraska. 2023. Parallel External Sorting of ASCII Records Using Learned Models. arXiv:2305.05671 [cs.DB] https://arxiv.org/abs/2305.05671

[46] Marius Kuhrt, Bernhard Seeger, Sebastian Wild, and Goetz Graefe. 2025. Adaptive Sorting for Large Keys, Strings, and Database Rows. In *BTW*.

[47] Laurens Kuiper. 2025. Redesigning DuckDB's Sort, Again. https://duckdb.org/2025/09/24/sorting-again.html

[48] Laurens Kuiper and Hannes Mühleisen. 2023. These Rows Are Made for Sorting and That's Just What We'll Do. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. 2050–2062.

[49] Sai Choi Kwan and Jean-Loup Baer. 1985. The I/O Performance of Multiway Mergesort and Tag Sort. *IEEE Trans. Comput.* C-34, 4 (1985), 383–387.

[50] P.A. Larson. 2003. External Sorting: Run Formation Revisited. *IEEE Transactions on Knowledge and Data Engineering* 15, 4 (2003), 961–972.

[51] Per-Åke Larson and Goetz Graefe. 1998. Memory Management during Run Generation in External Sorting. In *SIGMOD*. 472–483.

[52] Viktor Leis, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2014. Morsel-Driven Parallelism: A NUMA-aware Query Evaluation Framework for the Many-Core Age. In *SIGMOD*. 743–754.

[53] Viktor Leis and Maximilian Kuschewski. 2021. Towards Cost-Optimal Query Processing in the Cloud. *Proc. VLDB Endow.* 14, 9 (May 2021), 1606–1612.

[54] Rui Liu, Aaron J. Elmore, Michael J. Franklin, and Sanjay Krishnan. 2024. Riveter: Adaptive Query Suspension and Resumption Framework for Cloud Native Databases. In *ICDE*. 3975–3988.

[55] Gurmeet Singh Manku, Sridhar Rajagopalan, and Bruce G. Lindsay. 1998. Approximate Medians and Other Quantiles in One Pass and with Limited Memory. In *Proceedings of the 1998 ACM SIGMOD International Conference on Management of Data*. 426–435.

[56] C. Mohan and Inderpal Narang. 1992. Algorithms for Creating Indexes for Very Large Tables without Quiescing Updates. In *Proceedings of the 1992 ACM SIGMOD International Conference on Management of Data*. 361–370.

[57] Alex Monahan. 2025. Sorting on Insert for Fast Selective Queries. https://duckdb.org/2025/05/14/sorting-for-fast-selective-queries.html

Unpublished working draft. Not for distribution.

[58] Morwenn. 2025. Morwenn/Vergesort. https://github.com/Morwenn/vergesort

[59] J. Ian Munro and Sebastian Wild. 2018. Nearly-Optimal Mergesorts: Fast, Practical Sorting Methods That Optimally Adapt to Existing Runs. In *26th Annual European Symposium on Algorithms, ESA 2018, August 20-22, 2018, Helsinki, Finland*, Vol. 112. 63:1–63:16.

[60] Kihyeon Myung, Sunggon Kim, Heon Young Yeom, and Jiwoong Park. 2021. Efficient and Scalable External Sort Framework for Nvme SSD. *IEEE Trans. Comput.* 70, 12 (Dec. 2021), 2211–2217.

[61] Chris Nyberg, Tom Barclay, Zarka Cvetanovic, Jim Gray, and Dave Lomet. 1995. AlphaSort: A Cache-Sensitive Parallel External Sort. *The VLDB Journal* 4, 4 (Oct. 1995), 603–628.

[62] Saher Odeh, Oded Green, Zahi Mwassi, Oz Shmueli, and Yitzhak Birk. 2012. Merge Path - Parallel Merging Made Simple. In *2012 IEEE 26th International Parallel and Distributed Processing Symposium Workshops & PhD Forum*. 1611–1618.

[63] Riki Otaki, Jun Hyuk Chang, Charles Benello, Aaron J. Elmore, and Goetz Graefe. 2025. Resource-Adaptive Query Execution with Paged Memory Management. In *CIDR*.

[64] HweeHwa Pang, Michael J. Carey, and Miron Livny. 1993. Memory-Adaptive External Sorting. In *VLDB*. 618–629.

[65] Danila Piatov, Sven Helmer, Anton Dignös, and Fabio Persia. 2021. Cache-Efficient Sweeping-Based Interval Joins for Extended Allen Relation Predicates. *The VLDB Journal* 30, 3 (May 2021), 379–402.

[66] Aneesh Raman, Subhadeep Sarkar, Matthaios Olma, and Manos Athanassoulis. 2023. Indexing for Near-Sorted Data. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. 1475–1488.

[67] Maximilian Reif and Thomas Neumann. 2022. A Scalable and Generic Approach to Range Joins. *Proc. VLDB Endow.* 15, 11 (July 2022), 3018–3030.

[68] G. Saake, V. Linnemann, P. Pistor, and L. Wegner. 1989. Sorting, Grouping and Duplicate Elimination in the Advanced Information Management Prototype. In *Proceedings of the 15th International Conference on Very Large Data Bases*. 307–316.

[69] Betty Salzberg. 1989. Merging Sorted Runs Using Large Main Memory. *Acta Informatica* 27, 3 (Jan. 1989), 195–215.

[70] Johann Schleier-Smith, Vikram Sreekanti, Anurag Khandelwal, Joao Carreira, Neeraja J. Yadwadkar, Raluca Ada Popa, Joseph E. Gonzalez, Ion Stoica, and David A. Patterson. 2021. What Serverless Computing Is and Should Become: The next Phase of Cloud Computing. *Communications of The Acm* 64, 5 (April 2021), 76–84.

[71] Nisheeth Shrivastava, Chiranjeeb Buragohain, Divyakant Agrawal, and Subhash Suri. 2004. Medians and beyond: new aggregation techniques for sensor networks. In *Proceedings of the 2nd international conference on Embedded networked sensor systems*. 239–249.

[72] A. Silberschatz, H.F. Korth, and S. Sudarshan. 2020. *Database System Concepts*.

[73] Malte Skarupke. 2016. I Wrote a Faster Sorting Algorithm. https://probablydance.com/2016/12/27/i-wrote-a-faster-sorting-algorithm/

[74] Malte Skarupke. 2017. Faster Sorting Algorithm Part 2. https://probablydance.com/2017/01/17/faster-sorting-algorithm-part-2/

[75] D. Taniar and Wenny Rahayu. 2000. Sorting in Parallel Database Systems. In *HPC Asia*. 830–835 vol.2.

[76] The PostgreSQL Global Development Group. 2024. PostgreSQL. https://www.postgresql.org

[77] Gaurav Vaghasiya and Shiva Jahangiri. 2025. [Experiments & Analysis] Hash-Based vs. Sort-Based Group-By-Aggregate: A Focused Empirical Study [Extended Version]. arXiv:2411.13245 [cs]

[78] A.Inkeri Verkamo. 1989. Performance Comparison of Distributive and Mergesort as External Sorting Algorithms. *Journal of Systems and Software* 10, 3 (1989), 187–200.

[79] Lei Wei, Chuan Heng Foh, Bingsheng He, and Jianfei Cai. 2018. Towards Efficient Resource Allocation for Heterogeneous Workloads in IaaS Clouds. *IEEE Transactions on Cloud Computing* 6, 1 (2018), 264–275.

[80] Richard Wesley. 2023. DuckDB's AsOf Joins: Fuzzy Temporal Lookups. https://duckdb.org/2023/09/15/asof-joins-fuzzy-temporal-lookups.html

[81] Huanchen Zhang, Yihao Liu, and Jiaqi Yan. 2024. Cost-Intelligent Data Analytics in the Cloud. In *CIDR*.

[82] Weiye Zhang and Per-Åke Larson. 1997. Dynamic Memory Adjustment for External Mergesort. In *Proceedings of the 23rd International Conference on Very Large Data Bases*. 376–385.

[83] LuoQuan Zheng and P.-A. Larson. 1996. Speeding up External Mergesort. *IEEE Transactions on Knowledge and Data Engineering* 8, 2 (1996), 322–332.

Unpublished working draft. Not for distribution.